

Universidade Estadual de Maringá
Ciência da Computação

Relatório do Trabalho de Programação de Sistemas

Alunos:

Guilherme S. L. Lopes –
RA143630

Lucas Nakamura Batista –
RA143494

Professor:

Ronaldo A. L. Gonçalves

30 de junho de 2026

1 Funcionamento Geral e Modular

1.1 Funcionamento Geral

O trabalho realizado consiste em uma "calculadora" de determinantes de matrizes de qualquer ordem. Nosso programa funciona utilizando de uma leitura de um arquivo .txt com uma tabela de números, e ao selecionar uma ordem m , pegamos as m linhas e m colunas dessa tabela, e fazemos o cálculo do determinante da matriz resultante dessa seleção.

Após realizar a seleção da matriz, o cálculo do determinante é feito de forma simples, usando do Teorema de Laplace e da Regra de Sarrus. Ao jogar a matriz no software, ele compara a ordem, caso ela seja de um valor superior a quatro, será feito o cálculo usando o Teorema de Laplace, e caso seja menor, fará ou por Regra de Sarrus, ou caso for de ordem 2 ou 1, fará o cálculo do determinante usando da multiplicação da diagonal principal menos a diagonal secundária, ou retornará o número da matriz unidade.

Caso a matriz estiver na condição para calcular utilizando do Laplace, ela cria uma matriz resultante da exclusão da linha e da coluna da matriz original, e calcula o determinante dessa matriz determinante de forma recursiva, até chegar em uma matriz de ordem 3. Após calcular o determinante da matriz resultante, é feito o cálculo do cofator, multiplicando o determinante por -1 elevado a linha mais a coluna do elemento do cofator.

1.2 Funcionamento Modular

- `Matriz criaMatriz(int ordem);`
 - Recebe como parâmetro um inteiro representando a ordem da matriz, aloca dinamicamente na memória os espaços para a linha e coluna do tamanho da `ordem` e retorna essa matriz.
- `void alocaValoresMatriz(Matriz *matriz, int ordem);`
 - Recebe como parâmetro uma matriz vazia e a ordem dessa matriz, lê o arquivo "`matriz.txt`" que contém diversos valores diferentes que estão ordenados por linha e coluna, verifica a ordem da matriz e aloca os elementos de suas respectivas posições.
- `double calculaMatriz(Matriz m, int ordem);`
 - Recebe uma matriz e calcula seu determinante. Se a ordem for maior que 3, ela chama o cálculo por Teorema de Laplace. Se a ordem for 3, usa a Regra de Sarrus. Se for 2, aplica a fórmula direta $a_{11}a_{22} - a_{12}a_{21}$. Se for 1, retorna o único elemento da matriz.
- `double calculaSarrus(Matriz m);`
 - Define dois valores, um com a soma dos produtos das diagonais principais e outro com a soma dos produtos das diagonais secundárias.

$$\begin{cases} a = (a_{11} \cdot a_{22} \cdot a_{33}) + (a_{12} \cdot a_{23} \cdot a_{31}) + (a_{13} \cdot a_{21} \cdot a_{32}) \\ b = (a_{31} \cdot a_{22} \cdot a_{13}) + (a_{32} \cdot a_{23} \cdot a_{11}) + (a_{33} \cdot a_{21} \cdot a_{12}) \end{cases}$$

Depois retorna o valor da subtração de a por b ($a - b$)

- `double calculaLaplace(Matriz mat, int ordem);`
 - Cria uma matriz resultante da exclusão da linha i e da coluna j , após isso calcula o determinante dessa matriz resultante, multiplica pelo item a_{ij} da matriz original, e multiplica por $(-1)^{i+j}$.
- `Matriz matrizResultante(Matriz mat, int i, int j, int ordem);`
 - Cria a matriz resultante excluindo a linha i e a coluna j , mas mantendo os outros elementos.
- `void exibirMatriz(Matriz matriz, int ordem);`
 - Exibe a matriz selecionada pelo usuário formatada, com os números em suas respectivas linhas e colunas.
- `void esvaziaMemoria(Matriz matriz, int ordem);`
 - Esvazia a memória de uma matriz, percorrendo os vetores iniciais dando free neles, e no fim dando um free na matriz.

2 Atividades dos Participantes

No trabalho, as contribuições dos alunos Guilherme e Lucas foram as implementações das seguintes funções:

2.1 Guilherme dos Santos Lira Lopes

- `alocaValoresMatriz`
- `calculaMatriz`
- `exibirMatriz`
- `calculaSarrus`
- `esvaziaMemoria`

2.2 Lucas Nakamura Batista

- `criaMatriz`
- `calculaLaplace`
- `matrizResultante`
- `potencia`

Ao fim da implementação das funções, nos juntamos para corrigir quaisquer erros restantes, por exemplo o código não funcionava corretamente depois de uma certa ordem, ou ele retornava um valor próximo do resultado correto devido a diferença entre o float – o que estávamos usando até então – e o double.

3 Comentários Gerais

Nesse trabalho, nossa maior dificuldade foi na implementação do calculo de Laplace, visto que é um processo trabalhoso, e também a correção de alguns erros, como quando uma matriz era de uma ordem acima de 5, essa matriz não era alocada corretamente, e também o supramencionado erro de arredondamento do float, que deixava o resultado incorreto.

De certa forma não achamos que o trabalho tinha uma dificuldade tão elevada, com exceção de Laplace e na correção de erros, além disso devido a nossa implementação, o tempo de execução para calcular matrizes de ordem acima de 11 fica extremamente elevado e não é possível obter o resultado, essa é a única coisa que encontramos que não funciona dentro do nosso código.